

Week-7 & 8

Deployment of Java Web Application using Docker Container

1.Aim

To deploy a Java Maven web application inside a Docker container instead of installing Apache Tomcat directly on the server.

2.Tools & Technologies Required

- Java 17 – Application runtime
- Apache Maven – Build automation tool
- Jenkins – CI/CD automation server
- Docker – Containerization platform
- GitHub – Source code repository
- AWS EC2 Ubuntu – Cloud server environment

3.Theory

Docker is a containerization platform that allows applications to run in isolated environments called containers. Instead of installing Apache Tomcat directly on the server, Docker allows Tomcat and the application to be packaged together inside a container image. This ensures consistency, portability, and easier deployment across different environments.

4.Architecture

GitHub → Jenkins → Maven Build → Docker Image → Docker Container → Browser

5.Procedure

Step 1: Install Docker

```
sudo apt update
```

```
sudo apt install docker.io -y
```

```
sudo systemctl start docker
```

```
sudo systemctl enable docker
```

Step 2: Provide Jenkins Permission to Use Docker

```
sudo usermod -aG docker jenkins
```

```
sudo systemctl restart jenkins
```

Step 3: Add Dockerfile in Project Repository

Create a file named Dockerfile and add:

```
FROM tomcat:10.1-jdk17
```

```
RUN rm -rf /usr/local/tomcat/webapps/*  
COPY target/*.war /usr/local/tomcat/webapps/ROOT.war  
EXPOSE 8080  
CMD ["catalina.sh", "run"]
```

Step 4: Update Jenkinsfile for Docker Deployment

Include stages to build WAR, build Docker image, remove old container, and run new container with port mapping.

(For reference, fork my repo .it contains Jenkinsfile and Dockerfile.

Git repo: <https://github.com/syoganandh/spring-boot-maven-example-helloworld.git>)

Step 5: Open AWS Security Group Port

Allow inbound traffic on port 8090.

Step 6: Run Jenkins Pipeline

Click 'Build Now' in Jenkins to build Docker image and start container.

Step 7: Access Application

Open `http://<EC2-IP>:8090/` in browser.

6.Result

The Java web application was successfully containerized using Docker and deployed as a running container accessible through the browser.

7.Conclusion

Docker-based deployment simplifies application delivery by packaging dependencies and server configuration together, making deployments portable and efficient.

Installation and Configuration of Docker to Create Containers Using Different OS Images

1.Aim

To install Docker on an AWS Ubuntu instance and create containers using different operating system images such as Ubuntu, Alpine, and CentOS.

2.Tools Required

- Docker – Containerization platform
- AWS EC2 Ubuntu Instance – Host machine
- Linux Terminal – Command execution

3.Theory

Docker is a containerization platform that allows applications and operating system environments to run in isolated containers. Containers are lightweight, portable, and share the host system's kernel while maintaining separate environments. Docker uses images to create containers. An OS image is a minimal version of an operating system such as Ubuntu or Alpine, which can be used to run applications inside containers.

4.Architecture

AWS Ubuntu Host → Docker Engine → OS Images → Containers

5.Procedure

Step 1: Connect to AWS Ubuntu Instance

```
ssh ubuntu@<EC2-PUBLIC-IP>
```

Step 2: Install Docker

```
sudo apt update
```

```
sudo apt install docker.io -y
```

Step 3: Start and Enable Docker Service

```
sudo systemctl start docker
```

```
sudo systemctl enable docker
```

```
docker --version
```

Step 4: Allow Non-Root User to Run Docker

```
sudo usermod -aG docker ubuntu
```

```
newgrp docker
```

```
docker run hello-world
```

Step 5: Download OS Images

```
docker pull ubuntu  
docker pull alpine  
docker pull centos  
docker images
```

Step 6: Run Ubuntu Container

```
docker run -it ubuntu /bin/bash  
exit
```

Step 7: Run Alpine Linux Container

```
docker run -it alpine /bin/sh
```

Step 8: Run CentOS Container

```
docker run -it centos /bin/bash
```

Step 9: View Running Containers

```
docker ps  
docker ps -a
```

Step 10: Stop and Remove Containers

```
docker stop <container_id>  
docker rm <container_id>
```

Step 11: Remove OS Images

```
docker rmi ubuntu  
docker rmi alpine  
docker rmi centos
```

6.Result

Docker was successfully installed and configured on the AWS Ubuntu instance. Containers were created and executed using Ubuntu, Alpine, and CentOS images.

7.Conclusion

Docker allows multiple lightweight OS environments to run on a single host system. Containers provide isolation, portability, and efficiency, making Docker an essential tool in modern DevOps practices.

Docker

1.What is Docker?

Docker is a platform used to **develop, ship, and run applications in containers**.

A **container** packages:

- Application code
- Runtime
- Libraries
- Dependencies

So, the application runs the **same way everywhere** (developer laptop, server, cloud).

2.Why Docker is Needed

Before Docker:

“It works on my machine” problem
Different environments → different errors
Manual server setup required

With Docker:

Same environment everywhere
Easy deployment
Lightweight and fast

3.What is a Container?

A **container** is a lightweight, isolated environment where an application runs.

Think of it like a **mini-computer** with:

- Its own filesystem
- Its own processes
- Shared OS kernel

4.What is a Docker Image?

A **Docker Image** is a **blueprint** used to create containers.

Image = Application + Dependencies

Container = Running instance of Image

Component	Role
Docker Client	Command line tool (docker)
Docker Daemon	Background service that runs containers
Image	Template to create container
Container	Running application

5.Important Docker Terms

Term	Meaning
Dockerfile	Script to build an image
Image	Application template
Container	Running instance of image
Registry	Storage for images (Docker Hub)
Volume	Persistent storage
Port Mapping	Connect container port to host port

6.Installing Docker (Ubuntu)

```
sudo apt update
sudo apt install docker.io -y
sudo systemctl start docker
sudo systemctl enable docker
```

Check:
docker --version

7.Running First Container

```
docker run hello-world
```

This downloads a test image and runs it.

8.Downloading an Image

```
docker pull tomcat:10.1-jdk17
```

List images:
docker images

9.Running a Container

```
docker run -d -p 8090:8080 tomcat:10.1-jdk17
```

Option Meaning

-d Run in background
-p Map ports (host:container)

10.What is a Dockerfile?

A **Dockerfile** is a script that defines how a Docker image is built.

Example for Java WAR app:

```
FROM tomcat:10.1-jdk17
RUN rm -rf /usr/local/tomcat/webapps/*
```

```
COPY target/*.war /usr/local/tomcat/webapps/ROOT.war
EXPOSE 8080
CMD ["catalina.sh", "run"]
```

11.Building an Image:

```
docker build -t myapp .
```

12.Running Your App Container:

```
docker run -d -p 8090:8080 --name mycontainer myapp
```

Access in browser:

```
http://<server-ip>:8090
```

13.Viewing Running Containers:

```
docker ps
```

14.Stopping a Container:

```
docker stop mycontainer
```

15.Removing Container:

```
docker rm mycontainer
```

16.Removing Image:

```
docker rmi myapp
```

17.Docker vs Virtual Machine

Docker	Virtual Machine
Lightweight	Heavy
Fast startup	Slow startup
Shares OS kernel	Full OS
Uses fewer resources	Uses more resources

18.Benefits of Docker in DevOps

- Consistent deployment
- Easy scaling
- Works with Jenkins CI/CD
- Faster environment setup
- Portable applications

19. Docker in Your Project

In your CI/CD pipeline:

1. Jenkins builds WAR using Maven

2. Docker builds image with WAR
 3. Docker runs container
 4. App becomes accessible via browser
- So, Docker replaces **manual Tomcat server setup**.